

Part I

```

module part1(
    input Clk,
    input R,
    input S,
    output Q
);
    wire R_g, S_g;
    wire Qa, Qb /* synthesis keep */;

    // Gate the R and S inputs with Clk
    assign R_g = R & Clk;
    assign S_g = S & Clk;

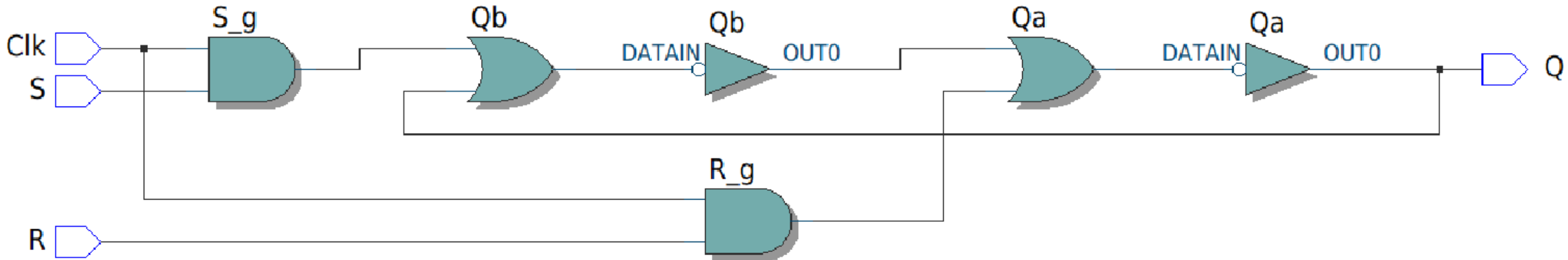
    // Cross-coupled NOR gates
    assign Qa = ~(R_g | Qb);
    assign Qb = ~(S_g | Qa);

    // Latch output
    assign Q = Qa;

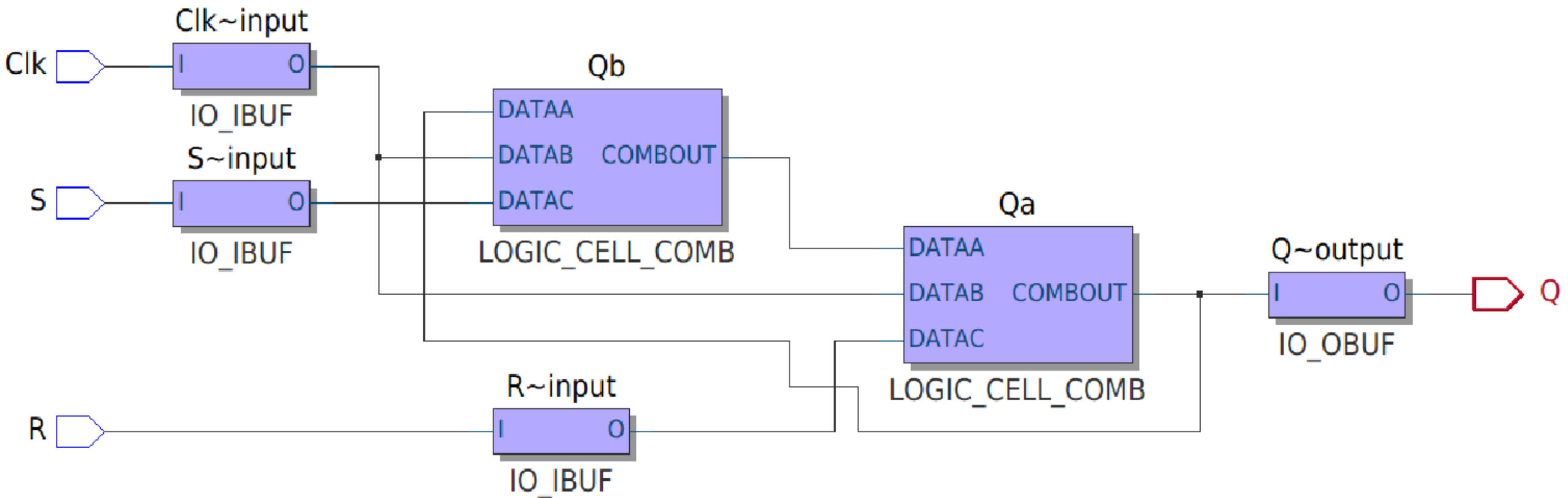
endmodule

```

RTL Schematic



TL Viewer

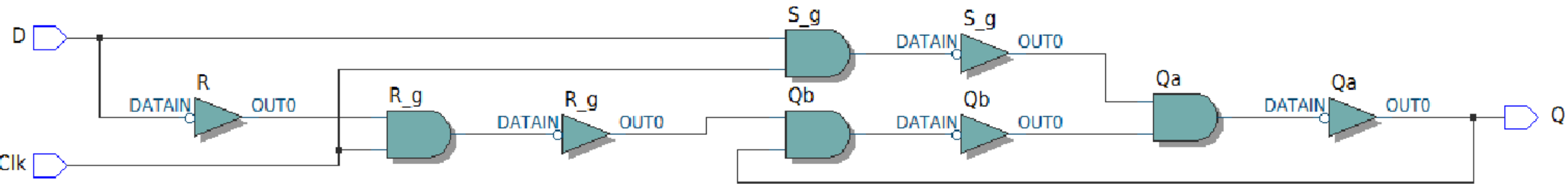


Part II

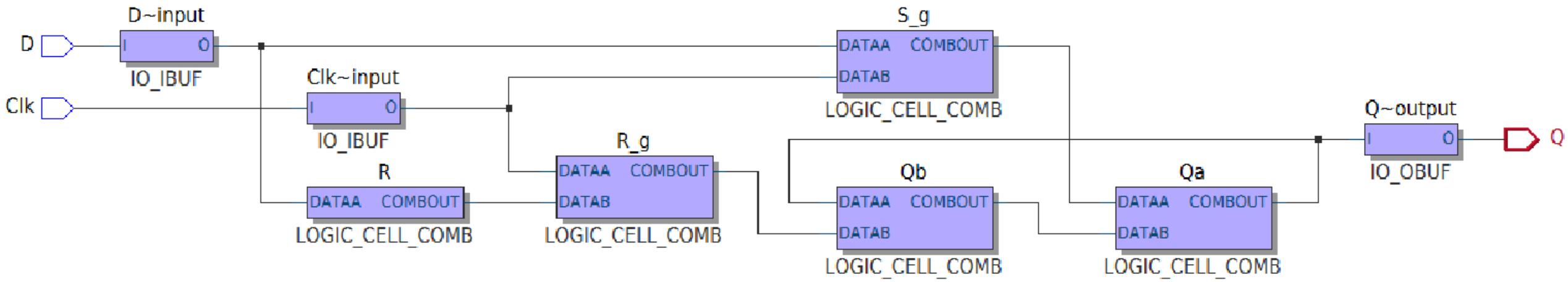
```
module part2(D, Clk, Q);  
  
  input D, Clk;  
  output Q;  
  
  wire R, S_g, R_g, Qa, Qb /* synthesis keep */;  
  
  // Generate S and R from D  
  assign R = ~D;  
  
  // Gated active-low S and R signals  
  assign S_g = ~(D & Clk);  
  assign R_g = ~(R & Clk);
```

```
// Cross-coupled NAND latch  
assign Qa = ~(S_g & Qb);  
assign Qb = ~(R_g & Qa);  
  
// Output  
assign Q = Qa;  
  
endmodule
```

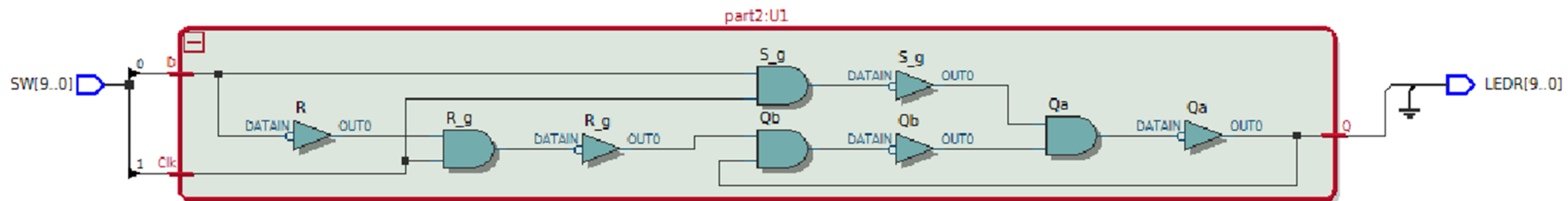
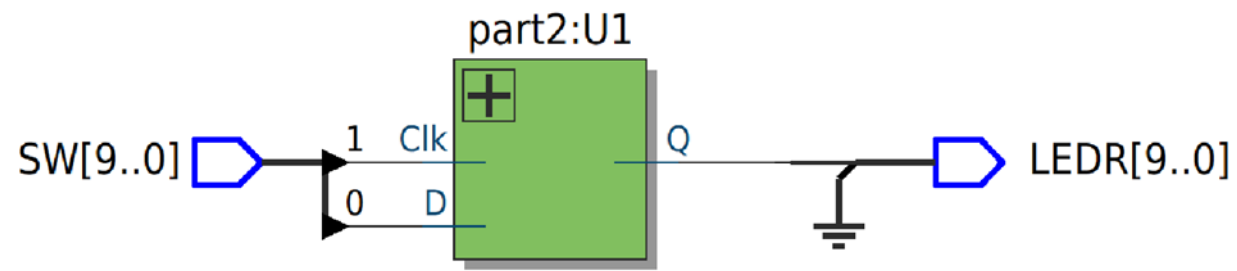
RTL Schematic



TL Viewer



Implementation on DE10Lite



Part III

```
module part3(
    input D,
    input Clock,
    output Q
);

    wire Qm;
    wire Clock_n;

    // Inverted clock for the master latch
    assign Clock_n = ~Clock;

    // Master latch
    part2 Master (
        .D (D),
        .Clk (Clock_n),
        .Q (Qm)
    );

    // Slave latch
    part2 Slave (
        .D (Qm),
        .Clk (Clock),
        .Q (Q)
    );

endmodule
```

```

module main (
    input [9:0] SW,
    output [9:0] LEDR
);

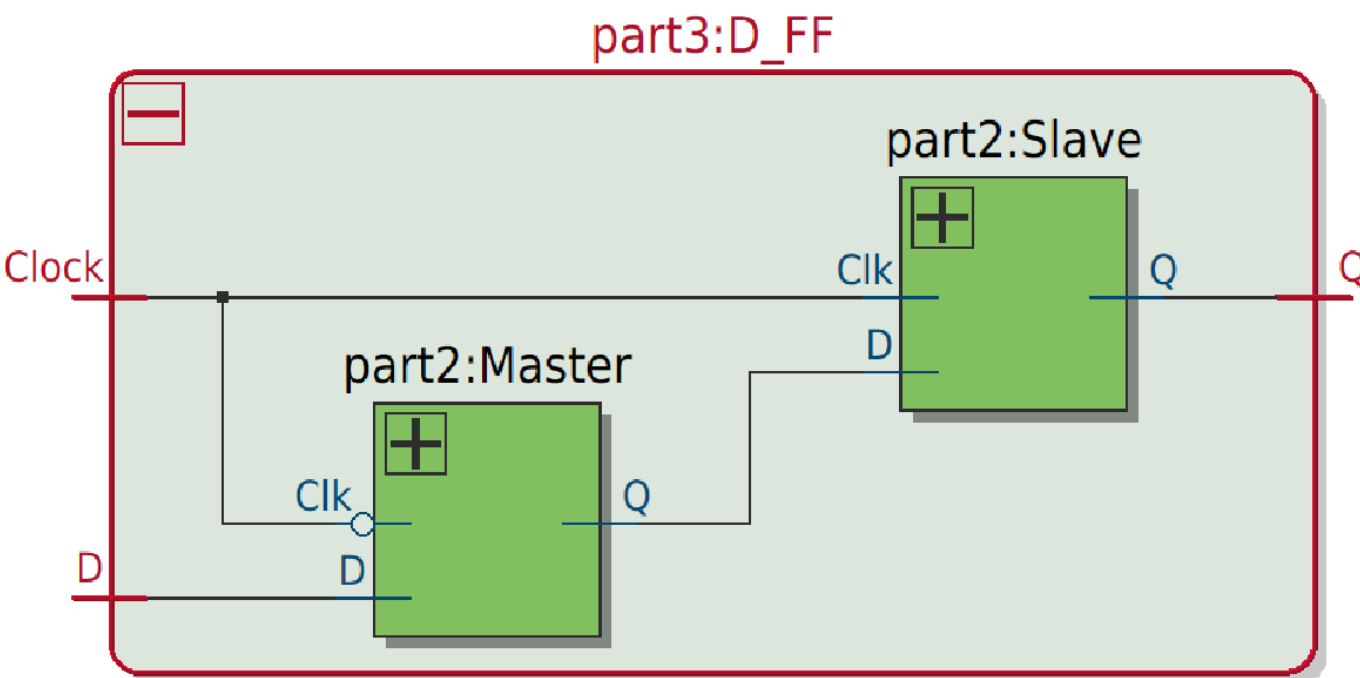
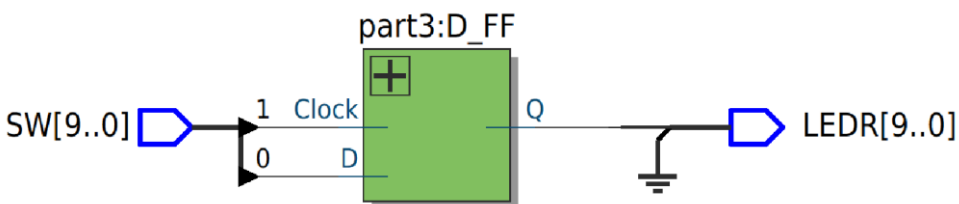
    wire Q;

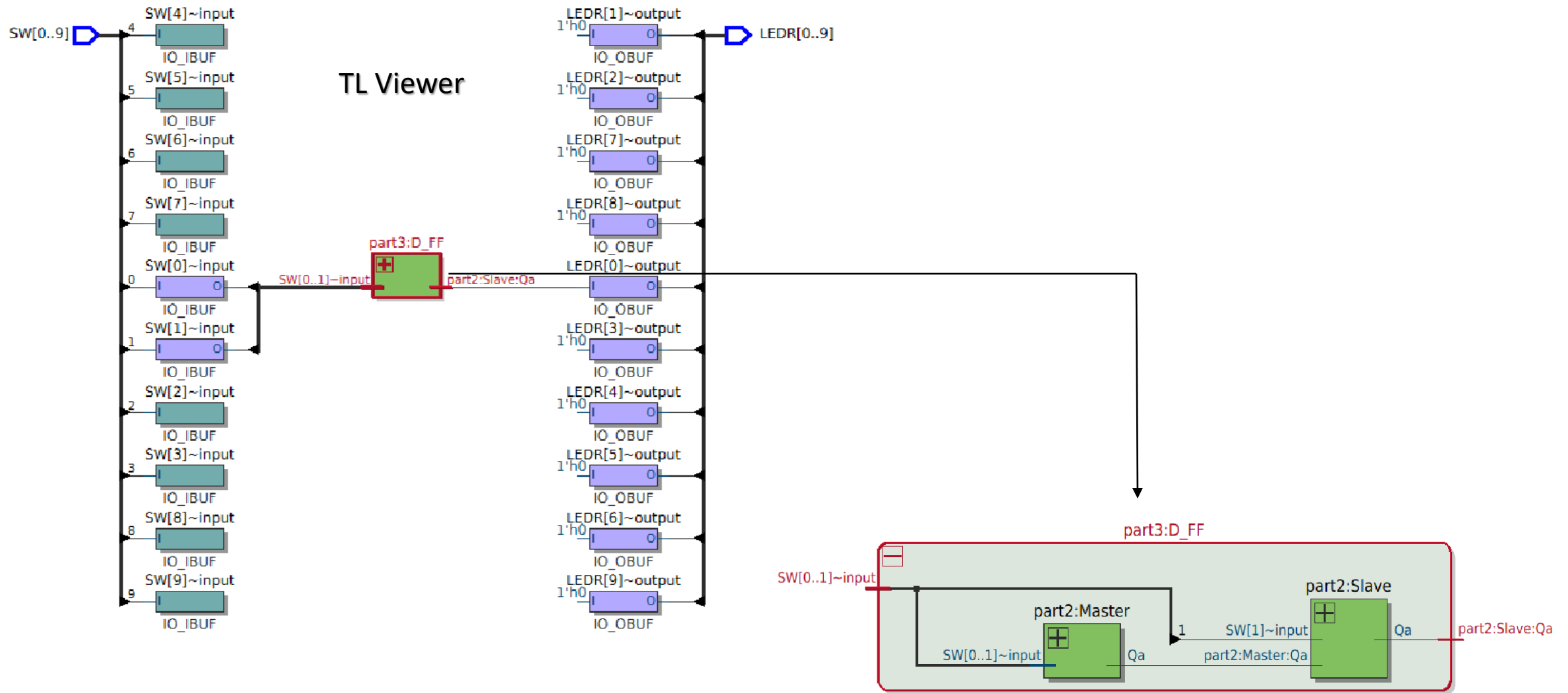
    part3 D_FF(
        .D    (SW[0]),
        .Clock (SW[1]),
        .Q     (Q)
    );

    assign LEDR[0] = Q;

endmodule

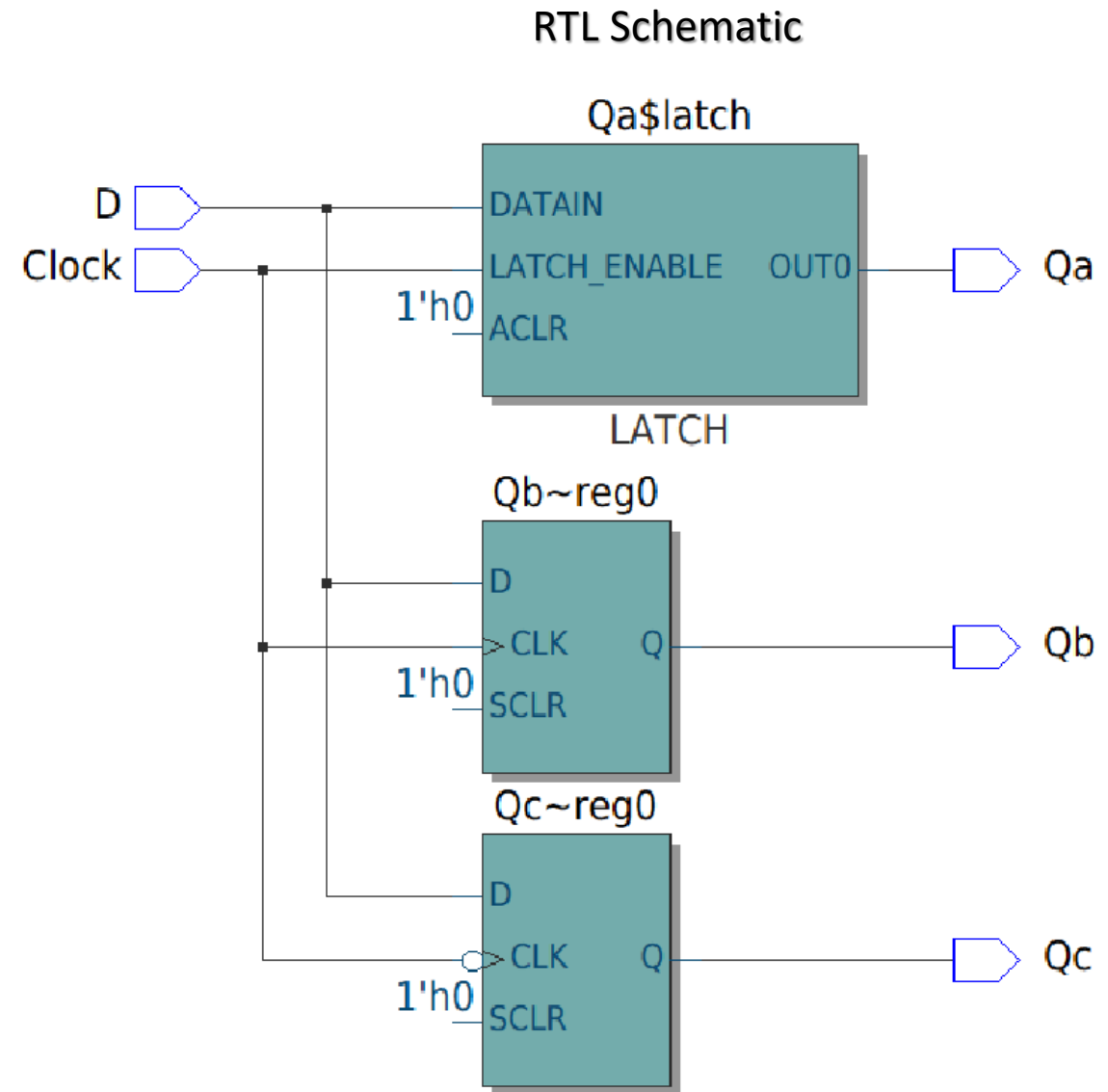
```

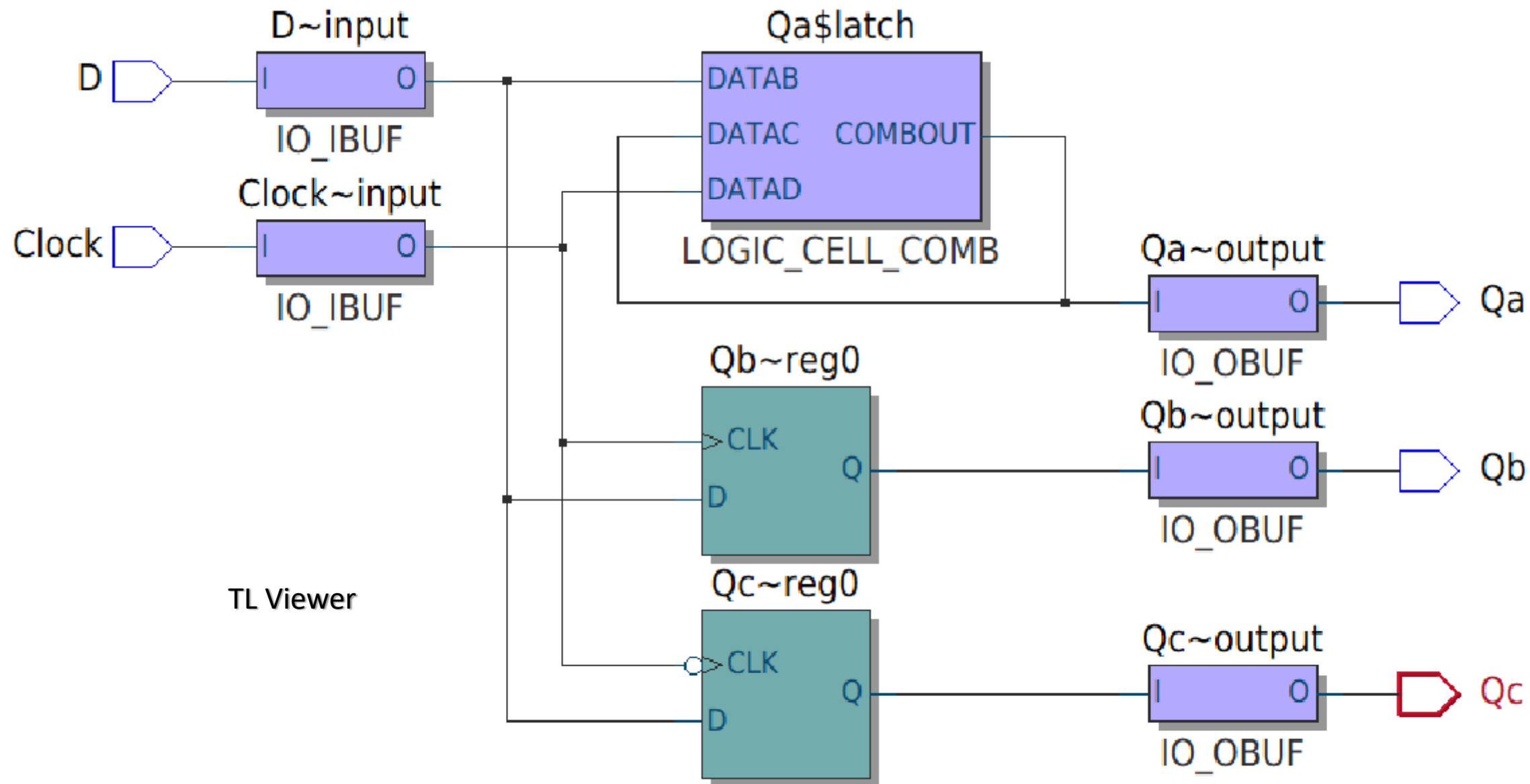




Part IV

```
module main (  
    input wire D,  
    input wire Clock,  
    output reg Qa,  
    output reg Qb,  
    output reg Qc  
);  
  
    // Gated D latch  
    // Transparent when Clock = 1  
    always @ (D or Clock)  
    begin  
        if (Clock)  
            Qa = D;  
    end  
  
    // Positive-edge-triggered D flip-flop  
    always @ (posedge Clock)  
    begin  
        Qb <= D;  
    end  
  
    // Negative-edge-triggered D flip-flop  
    always @ (negedge Clock)  
    begin  
        Qc <= D;  
    end  
  
endmodule
```





Part V

```
module main (  
    input [7:0] SW,  
    input [1:0] KEY,  
    output [7:0] HEX0,  
    output [7:0] HEX1,  
    output [7:0] HEX2,  
    output [7:0] HEX3  
);  
  
reg [7:0] A;  
  
// Store A  
// KEY0: active-low asynchronous reset  
// KEY1: clock  
always @(posedge KEY[1] or negedge KEY[0]) begin  
    if (!KEY[0])  
        A <= 8'b00000000;  
    else  
        A <= SW[7:0];  
end
```

```
// Stored A -> HEX3 and HEX2  
seg7_display display_A_high (  
    .bin_number(A[7:4]),  
    .seg_display(HEX3)  
);  
  
seg7_display display_A_low (  
    .bin_number(A[3:0]),  
    .seg_display(HEX2)  
);  
  
// Current switch value B -> HEX1 and HEX0  
seg7_display display_B_high (  
    .bin_number(SW[7:4]),  
    .seg_display(HEX1)  
);  
  
seg7_display display_B_low (  
    .bin_number(SW[3:0]),  
    .seg_display(HEX0)  
);  
  
endmodule
```

RTL

